THE CRUX: HOW TO DEVELOP SCHEDULING POLICY

How should we develop a basic framework for thinking about scheduling policies? What are the key assumptions? What metrics are important? What basic approaches have been used in the earliest of computer systems?

Workload Assumptions

↳ We make below simplifying assumptions about the running processes (jobs) in the system i.e workload

1. Each job runs for the same time
2. All jobs arrive at the same time
3. Once started, each job runs to completion
4. All jobs only use CPU (i.e they perform no I/O)
5. The run-time of each job is known

↳ These assumptions are be unrealistic but we will slowly relax them.

Scheduling metrics

↳ This enables us to compare different processes

Turnaround time:

The time at which the job completes minus the time at which the job arrived in the system.

$$T_{turnaround} = T_{completion} - T_{arrival}$$

↳ As we assumed at jobs arrive at the same time, So, $T_{arrival} = 0$ ∴

$$T_{turnaround} = T_{completion}$$

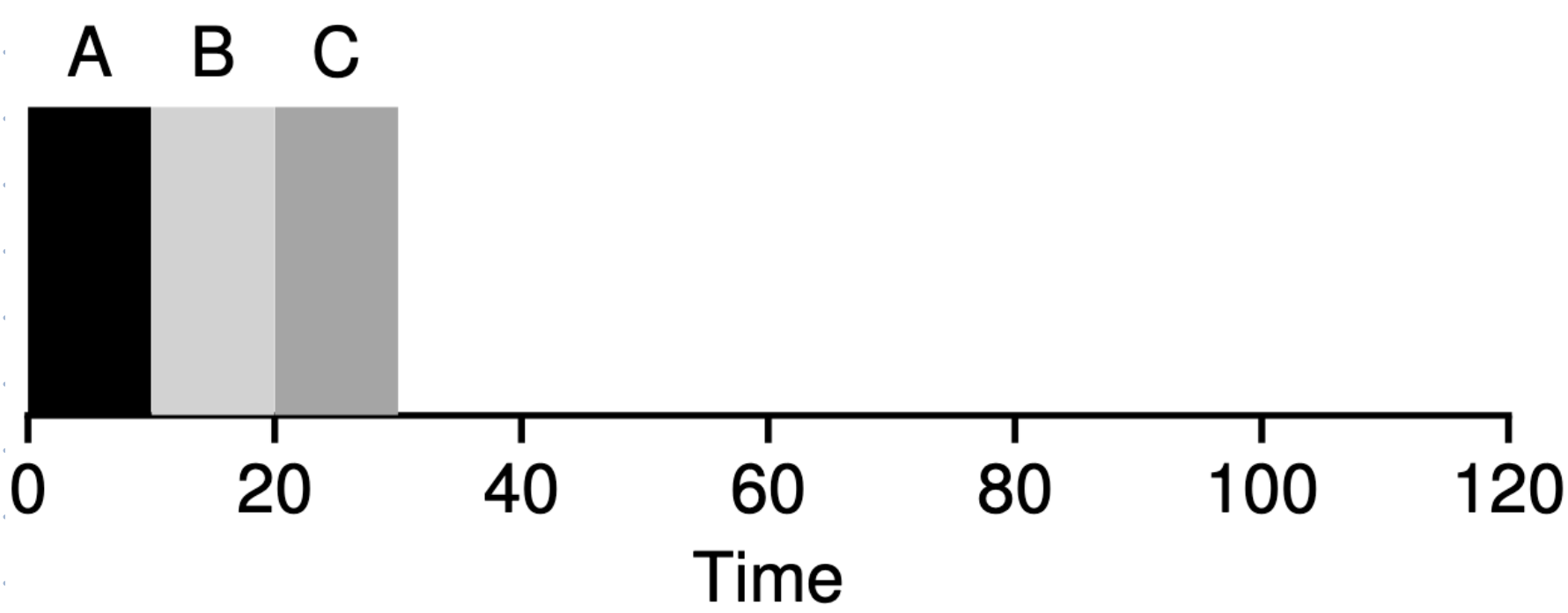
First In, First Out (FIFO)

- It is simple, easy to implement, & works well under our assumptions

- Imagine three jobs arrive in the system A, B & C at the same time ($T_{arrival} = 0$).

  Because FIFO has put some jobs first, lets assume that while they all arrived simultaneously, A arrived just a hair before B & B before C.

  Also, each job runs for 10 seconds.



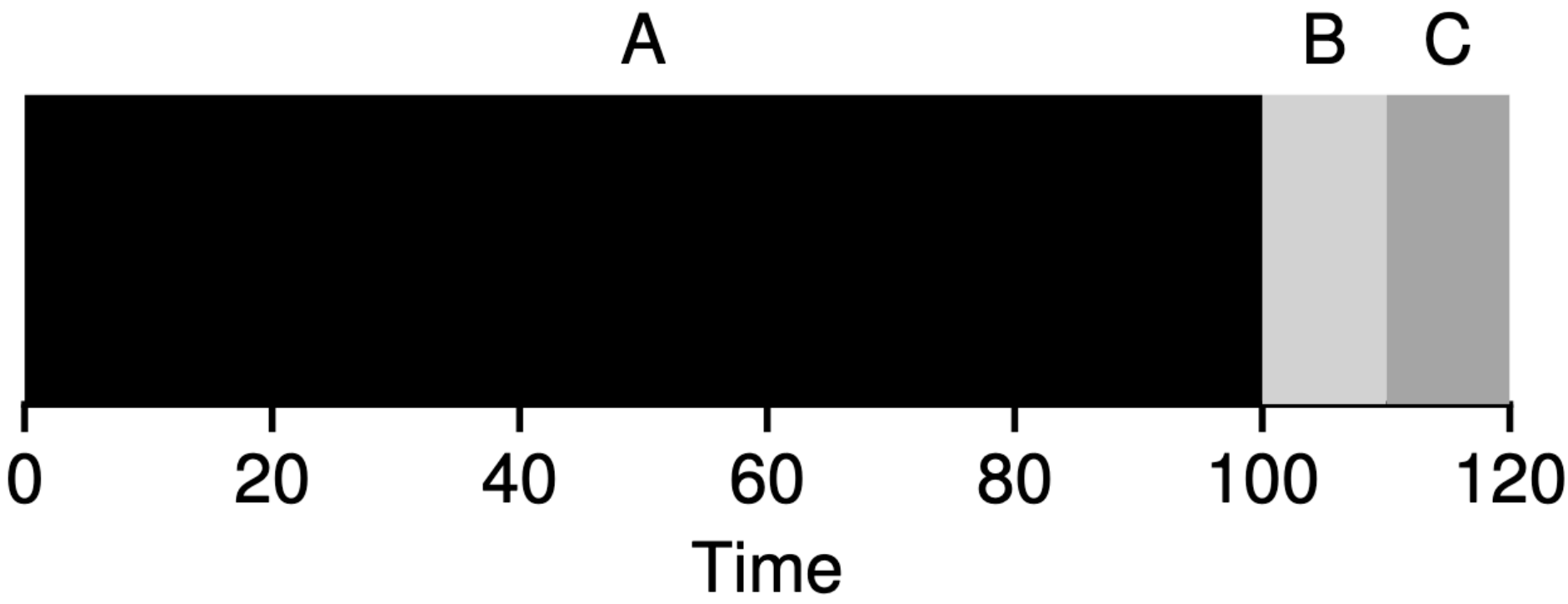
- Job A finishes at 10, B at 20 & C at 30. Thus, average turnaround time is:

$$\text{Avg } T_{turnaround} = \frac{10 + 20 + 30}{3}$$

$$= 20$$

- Now, lets relax assumption 1 i.e each jobs runs for the same time.

- lets imagine the same above but job A runs for 100, B & C for 10



- Job A finishes at 100, B at 110 & C at 120. Thus, the average turnaround time is:

$$\text{Avg } T_{\text{turnaround}} = \frac{100 + 110 + 120}{3}$$

$$= 110$$

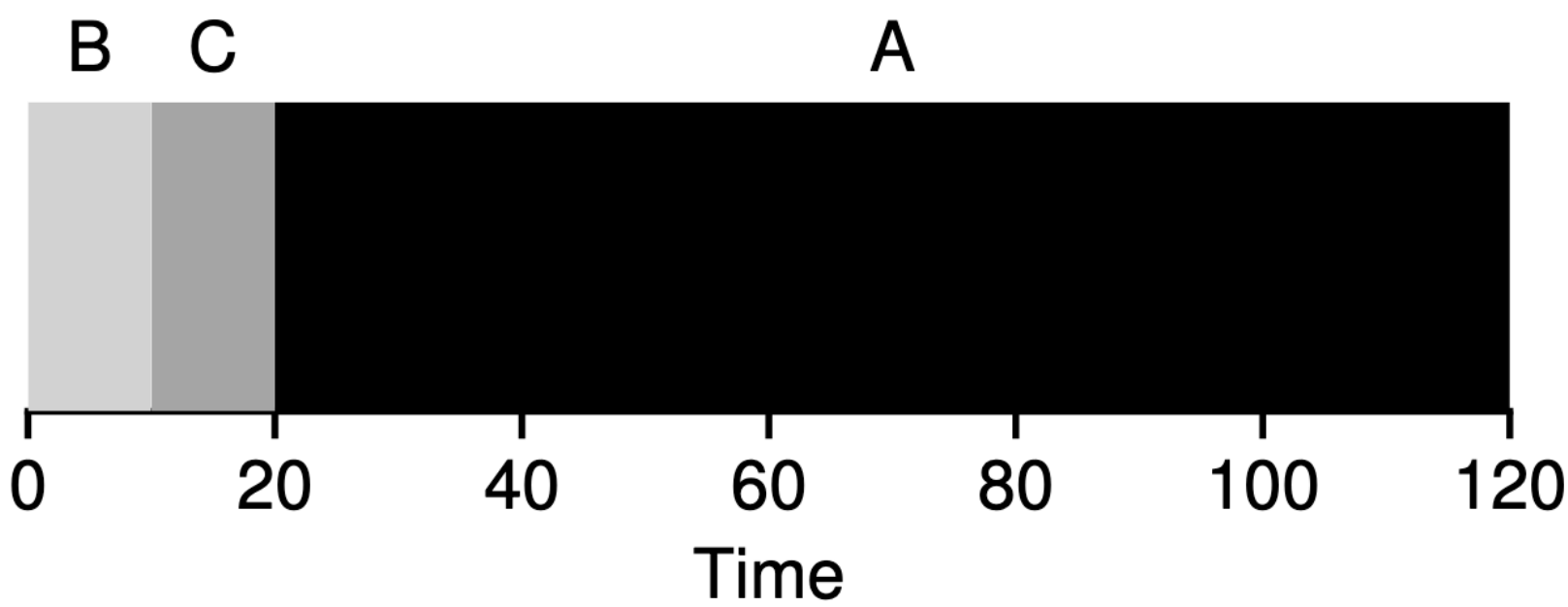
- This problem is known as the convoy effect, where a number of relatively short potential consumers of a resource get queued behind a heavy-weight consumer.

Analogy:

- The person infront of you at a grocery store counter has two full carts while you have a single bottle of water.

Shortest Job First (SJF)

- It runs the shortest job first then the next shortest and so on.



↳ Job A finishes at 10, B at 20 & C at 120. Thus, the avesage tusnasound time is:

$$\text{Avg } T_{\text{turnaround}} = \frac{10+20+120}{3}$$

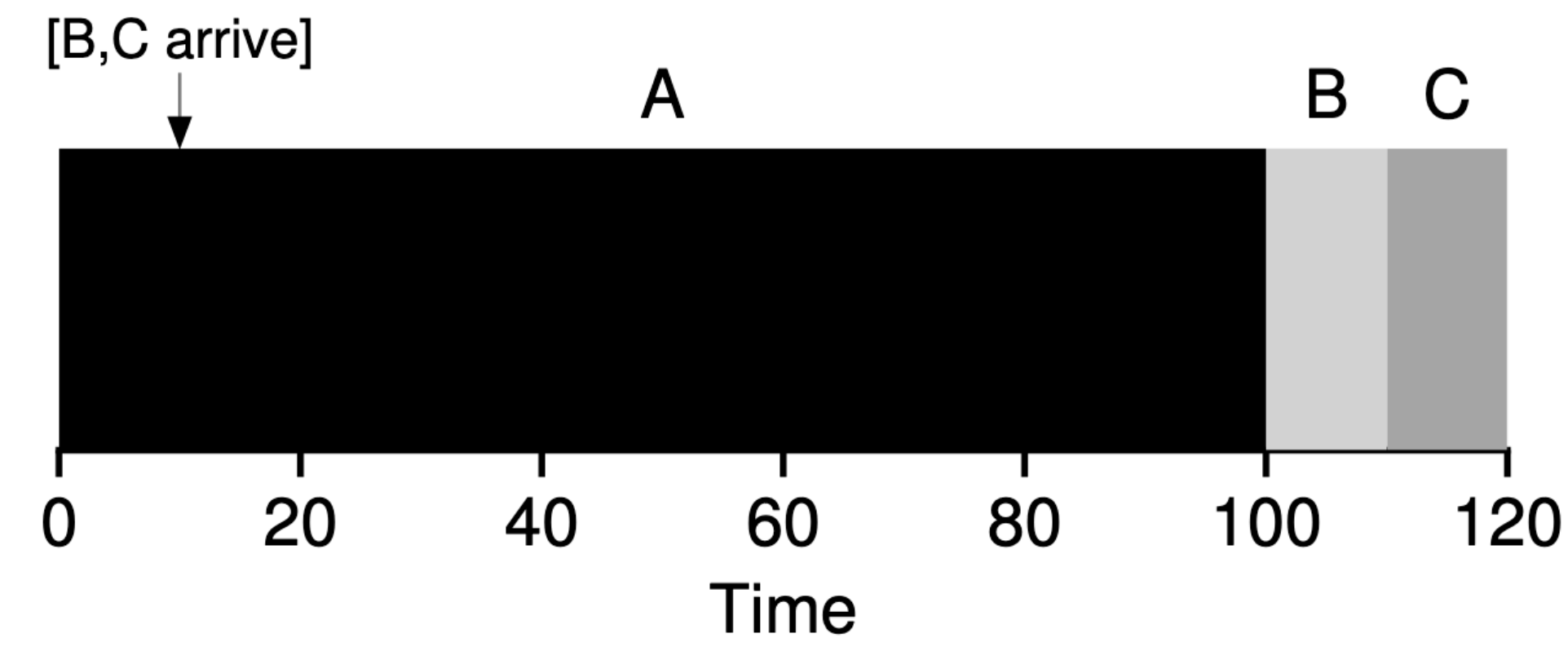
$$= 50$$

ASIDE: PREEMPTIVE SCHEDULERS

In the old days of batch computing, a number of **non-preemptive** schedulers were developed; such systems would run each job to completion before considering whether to run a new job. Virtually all modern schedulers are **preemptive**, and quite willing to stop one process from running in order to run another. This implies that the scheduler employs the mechanisms we learned about previously; in particular, the scheduler can perform a **context switch**, stopping one running process temporarily and resuming (or starting) another.

↳ Now, lets selax assumption 2 aswell i.e each job assives at the same time.

↳ lets imagine the same above but job A assives at t=0, A suns for 100s, B & C assive at t=10 & each sun for 10s.



↳ Since B & C assive after A & each job must sun to completion & so B&C get pushed to the end.

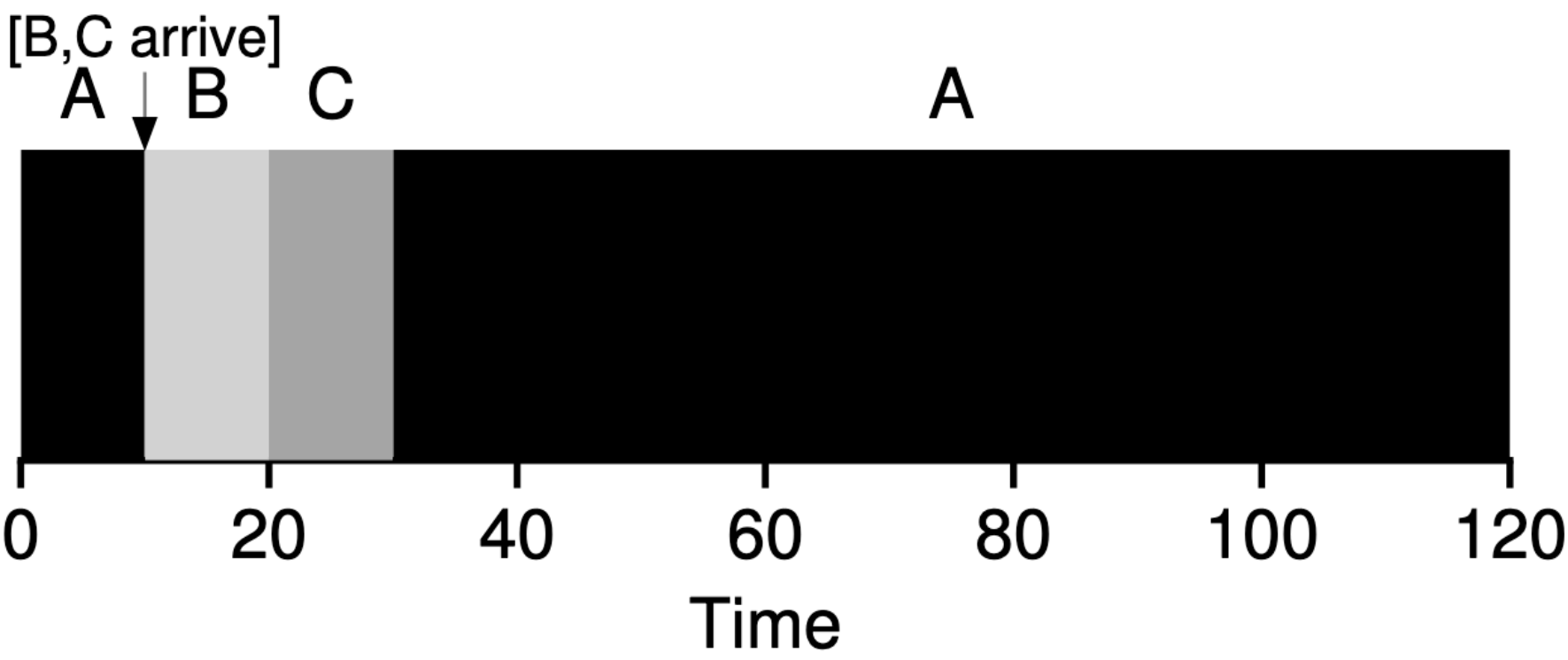
↳ Job A finishes at 100, B at 110 & C at 120. Thus, the avesage tusnasound time is:

$$\text{Avg } T_{turnaround} = \frac{100 + (110-10) + (120-10)}{3}$$

$$= 103.33$$

Shortest time to completion first (STCF)

- To address this concern, we need to relax assumption 3 i.e each job must run to completion.
- The scheduler performs context switching & prempts (kicks out) job A & decide to run on another job
- STCF is like SJF but with prempton (context switching)
- Any time a new job enters the system, the STCF determines which of jobs have the least time left, and schedules that one.



So, in our example, STCF prempts A & runs B&C to completion; then run job A.

Thus, the average turnaround time is:

$$\text{Avg } T_{turnaround} = \frac{(120-0) + (20-10) + (30-10)}{3}$$

$$= 50$$

A New metric: Response Time:

- For batch systems, non-preemptive schedulers make sense as program were run non-interactively as users submitted their jobs (eg punch cards) & came back later to collect it.

- Time-shared systems where users sit at the terminal and send commands to interact with the system, responsiveness is important.

$$T_{response} = T_{firstrun} - T_{arrival}$$

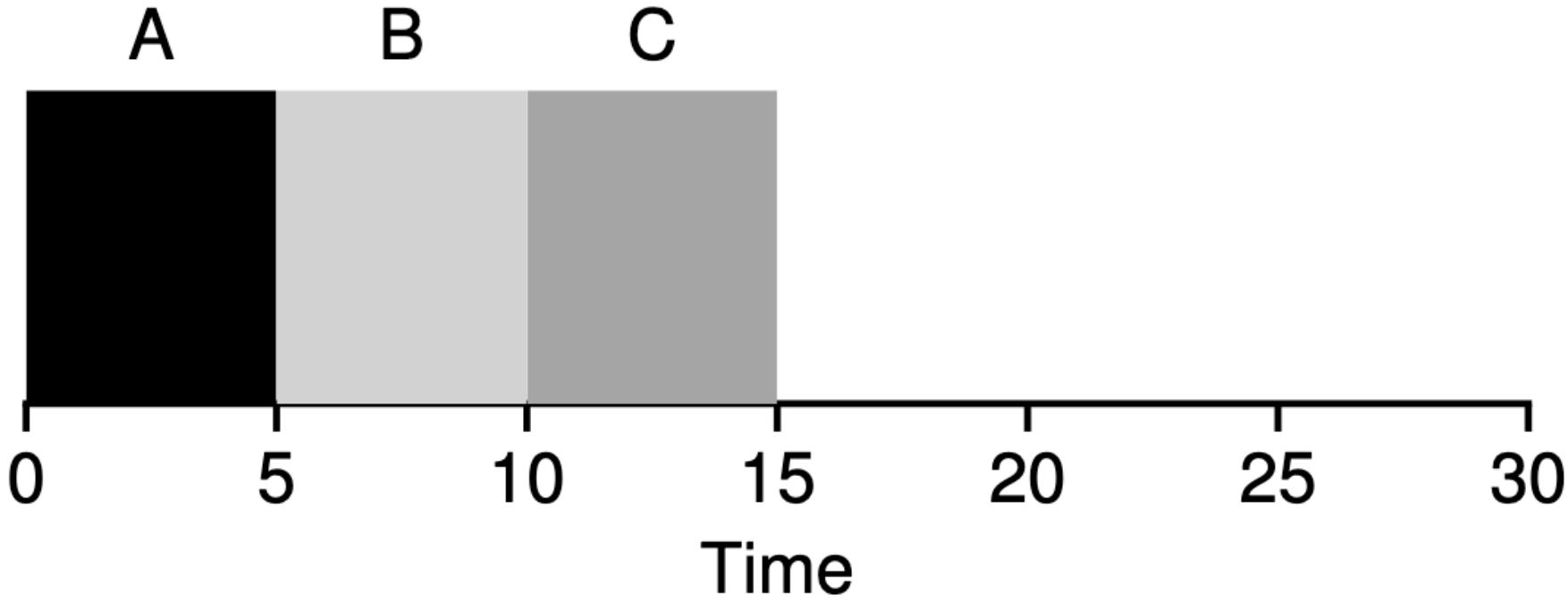


Figure 7.6: SJF Again (Bad for Response Time)

- Here, job C runs last & the user has to wait for Job A & B to finish before seeing output of job C.

Round Robin (RR)

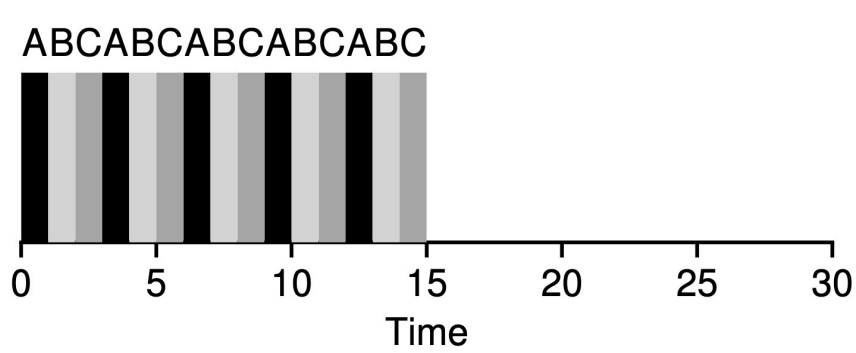
- RR runs a job for a time slice (scheduling quantum) & switches to another job in the queue.

- Assume Jobs A, B & C arrive at the same time & run for 5 seconds each.

- SJF runs each job to completion:

$$\text{Avg } T_{response} = \frac{0+5+10}{3}$$

$$= 5$$



- RR with a time slice of 1 second would cycle through jobs quickly,

$$\text{Avg Tresponse} = \frac{0+1+2}{3} = 1 \qquad \text{Avg Tturnaround} = \frac{13+14+15}{3} = 14$$

- Time slice needs to be long enough to minimize the cost of switching without making it so long that the system is no longer responsive
- RR is very bad at turnaround time
- Any policy that is fair (such as RR) divides CPU evenly among active processes will perform poorly on metrics such as turnaround time.

Incorporating I/O

- Relax assumption 4 i.e All programs do no I/O
- When a job requests for I/O, it is blocked & the scheduler should schedule an another job as CPU is idle during this time.
- Assume we have Jobs A & B that run for 50 ms but Job A does I/O after every 10 ms.
- So, A scheduler would run Job A first & then Job B

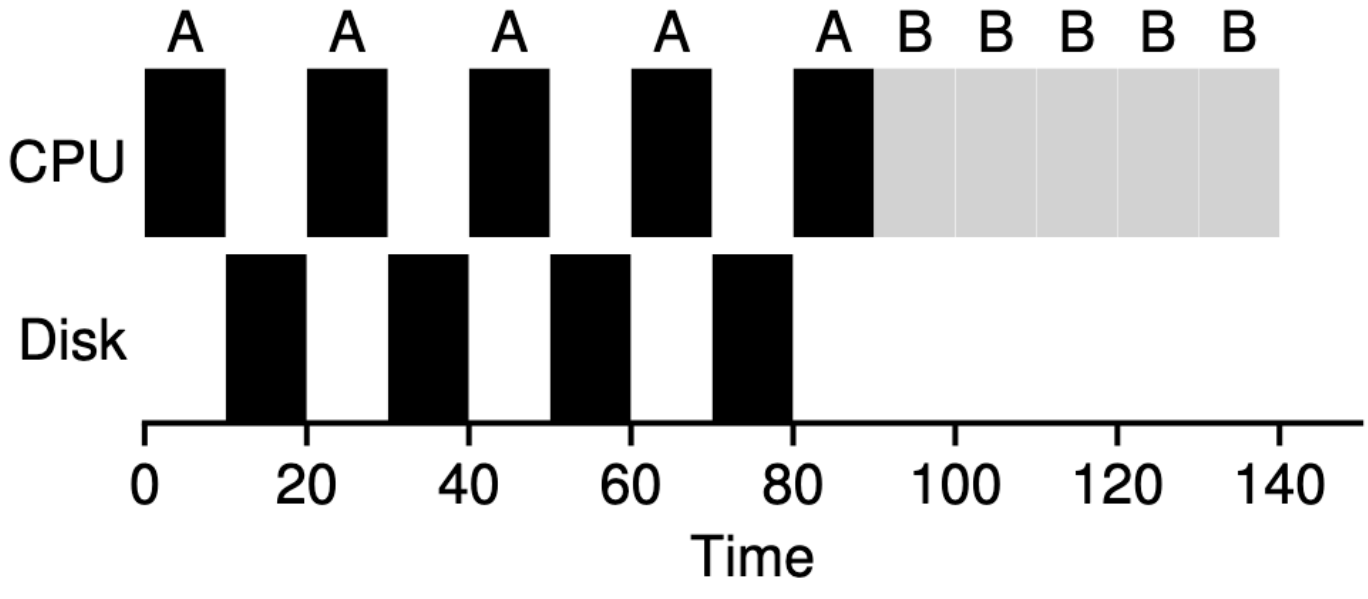


Figure 7.8: **Poor Use Of Resources**

↳ Instead, we treat Job A till I/O as an independent 10 ms sub-job

↳ STCF will run A first (as it is the shortest). Then, when 1st sub job of A completes, only B is left, and it begins running. Then, a new subjob of A is submitted, and it prempts B and runs A for 10 ms. So on.

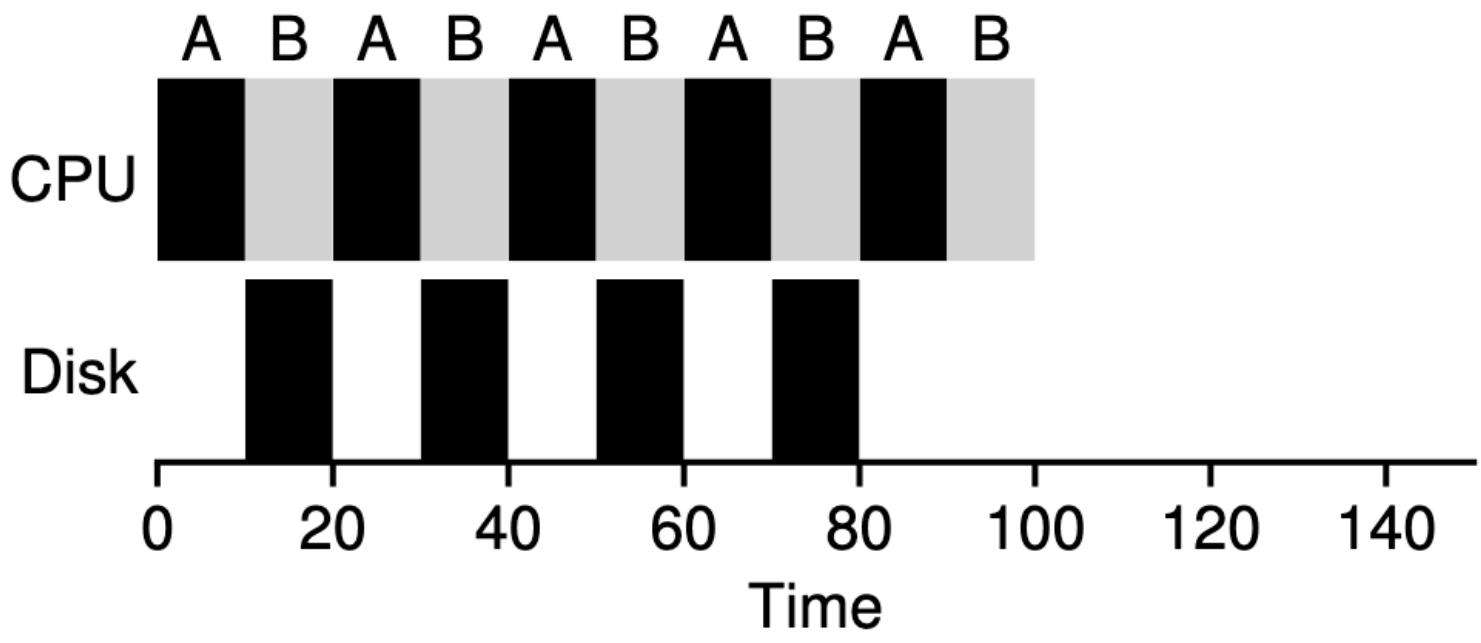


Figure 7.9: **Overlap Allows Better Use Of Resources**